

Machine-Level Programming I: Basics

15-213/18-213/15-213: Introduction to Computer Systems
5th Lecture, September 12, 2017

Today's Instructor:

Phil Gibbons

程序的转换与机器级表示

- **主要教学目标**
 - 了解高级语言与汇编语言、汇编语言与机器语言之间的关系
 - 掌握有关指令格式、操作数类型、寻址方式、操作类型等内容
 - 了解高级语言源程序中的语句与机器级代码之间的对应关系
 - 了解复杂数据类型（数组、结构等）的机器级实现
- **主要教学内容**
 - 介绍C语言程序与机器级指令之间的对应关系。
 - 主要包括：程序转换概述、IA-64指令系统、C语言中控制语句和过程调用等机器级实现、复杂数据类型（数组、结构等）的机器级实现等。
 - 本章所用的机器级表示主要以汇编语言形式表示为主。

采用逆向工程方法！

程序的机器级表示

- 分以下五个部分介绍

- **第一讲：程序转换概述**

- 机器指令和汇编指令
- 机器级程序员感觉到的属性和功能特性
- 高级语言程序转换为机器代码的过程

- **第二讲：IA-32 /x86-64指令系统**

- **第三讲：C语言程序的机器级表示**

- 选择语句的机器级表示
- 循环结构的机器级表示
- 过程调用的机器级表示

- **第四讲：复杂数据类型的分配和访问**

- 数组的分配和访问
- 结构体数据的分配和访问
- 联合体数据的分配和访问
- 数据的对齐

- **第五讲：越界访问和缓冲区溢出**

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

“指令”的概念

- 计算机中的指令有**微指令**、**机器指令**和**伪（宏）指令**之分
- **微指令**是微程序级命令，属于硬件范畴
- **伪指令**是由若干机器指令组成的指令序列，属于软件范畴
- **机器指令**介于二者之间，处于硬件和软件的交界面
 - 本章中提及的指令都指机器指令
- **汇编指令**是机器指令的汇编表示形式，即符号表示
- 机器指令和汇编指令一一对应，它们都与具体机器结构有关，都属于**机器级指令**

机器级指令

- 机器指令和汇编指令一一对应，都是机器级指令
- 机器指令是一个0/1序列，由若干**字段**组成

补码**11111010**的
的真值为多少？

100010 DW	mod	reg	r/m	disp8
100010 0 0	01	001	001	11111010

操作码

立即数(位移量)

寻址方式

寄存器编号

- 汇编指令是机器指令的符号表示 (**可能有不同的格式**)

mov [bx+di-6], cl 或 **movb %cl, -6(%bx,%di)**

Intel格式

AT&T 格式

mov、movb、bx、%bx等都是**助记符**

指令的功能为: **$M[R[bx] + R[di] - 6] \leftarrow R[cl]$**

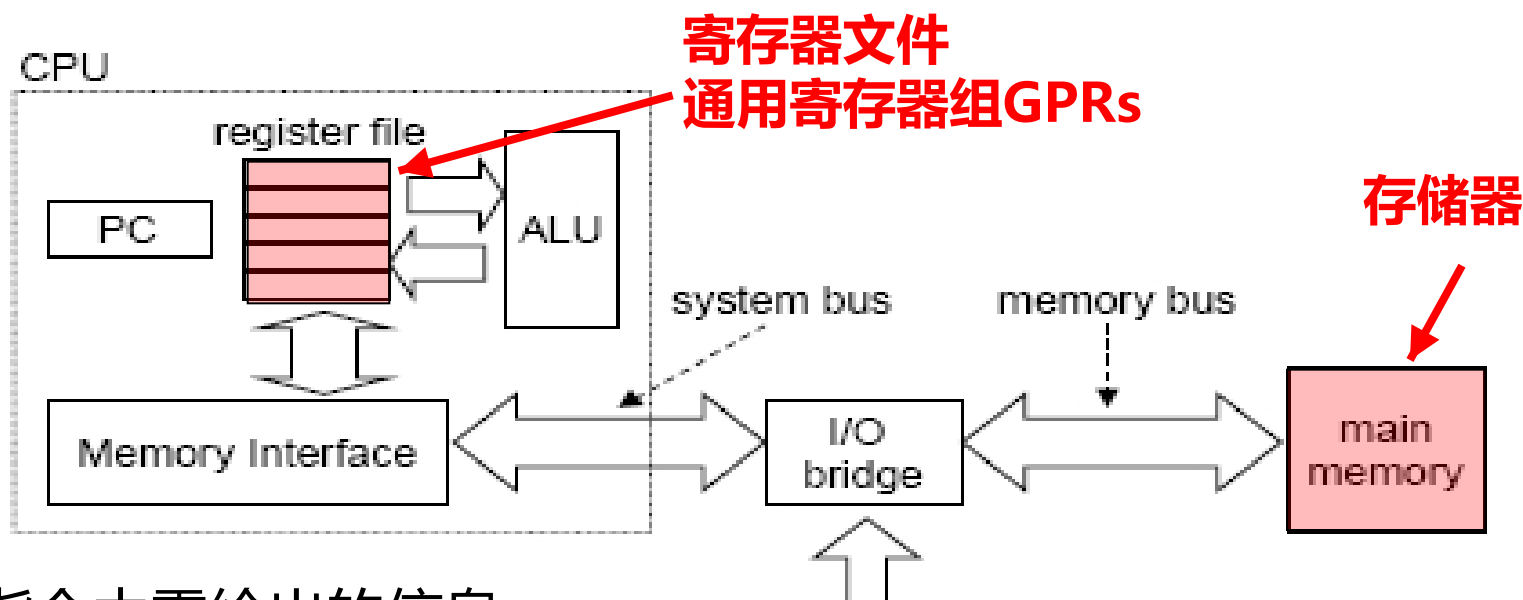
R: 寄存器内容

M: 存储单元内容

寄存器传送语言 **RLT** (Register Transfer Language)

计算机中数据的存储

- 计算机中的数据存放在哪里？



指令中需给出的信息：

操作性质（操作码）

源操作数1 或/和 源操作数2 （立即数、寄存器编号、存储地址）

目的操作数地址 （寄存器编号、存储地址）

存储地址的描述与操作数的数据结构有关！

指令集体系结构ISA

- ISA (Instruction Set Architecture) 位于软件和硬件之间
- 硬件的功能通过ISA提供出来
- 软件通过ISA规定的“指令”使用硬件
- ISA规定了：
 - 可执行的指令的集合，包括指令格式、操作种类以及每种操作对应的操作数的相应规定；
 - 指令可以接受的操作数的类型；
 - 操作数所能存放的寄存器组的结构，包括每个寄存器的名称、编号、长度和用途；
 - 操作数所能存放的存储空间的大小和编址方式；
 - 操作数在存储空间存放时按照大端还是小端方式存放；
 - 指令获取操作数的方式，即寻址方式；
 - 指令执行过程的控制方式，包括程序计数器、条件码定义等。

Today: Machine Programming I: Basics

- History of Intel processors and architectures
- **Assembly Basics: Registers, operands, move**
- Arithmetic & logical operations
- C, assembly, machine code

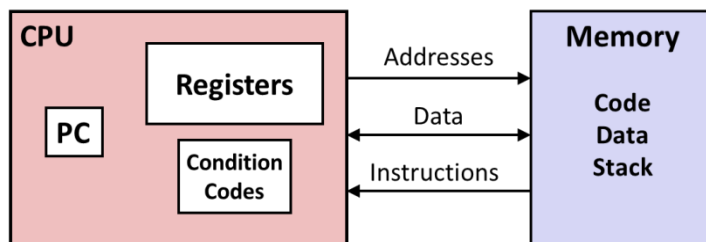
Levels of Abstraction

C programmer

C code

**Nice clean layers,
but beware...**

Assembly programmer



Computer Designer

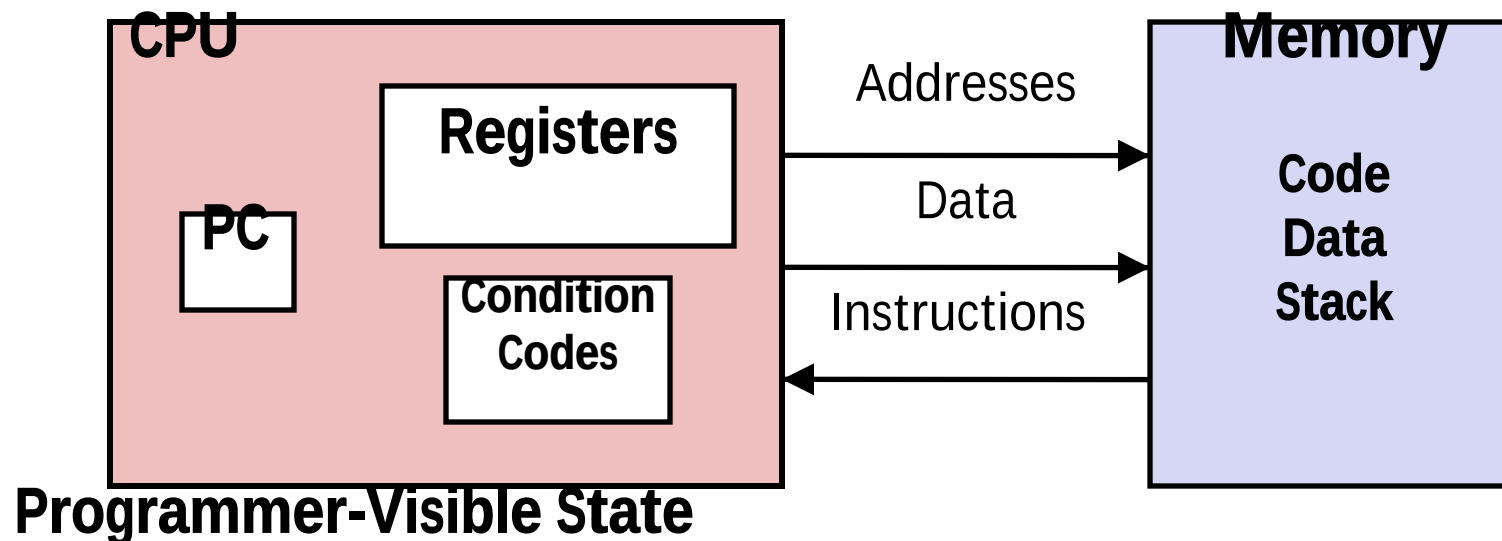
Caches, clock freq, layout, ...

Of course, you know that: It's why you are taking this course.

Definitions

- **Architecture:** (also ISA: instruction set architecture) The parts of a processor design that one needs to understand for writing correct machine/assembly code
 - Examples: instruction set specification, registers
 - **Machine Code:** The byte-level programs that a processor executes
 - **Assembly Code:** A text representation of machine code
- **Microarchitecture: Implementation of the architecture**
 - Examples: cache sizes and core frequency
- **Example ISAs:**
 - Intel: x86, IA32, Itanium, x86-64
 - ARM: Used in almost all mobile phones
 - RISC V: New open-source ISA

Assembly/ Machine Code View



- **PC: Program counter**

- Address of next instruction
- Called "RIP" (x86-64)

- **Register file**

- Heavily used program data

- **Condition codes**

- Store status information about most recent arithmetic or logical operation
- Used for conditional branching

- **Memory**

- Byte addressable array
- Code and user data
- Stack to support procedures

Assembly Characteristics: Data Types

- **“Integer” data of 1, 2, 4, or 8 bytes**
 - Data values
 - Addresses (untyped pointers)
- **Floating point data of 4, 8, or 10 bytes**
- **(SIMD vector data types of 8, 16, 32 or 64 bytes)**
- **Code: Byte sequences encoding series of instructions**
- **No aggregate types such as arrays or structures**
 - Just contiguously allocated bytes in memory

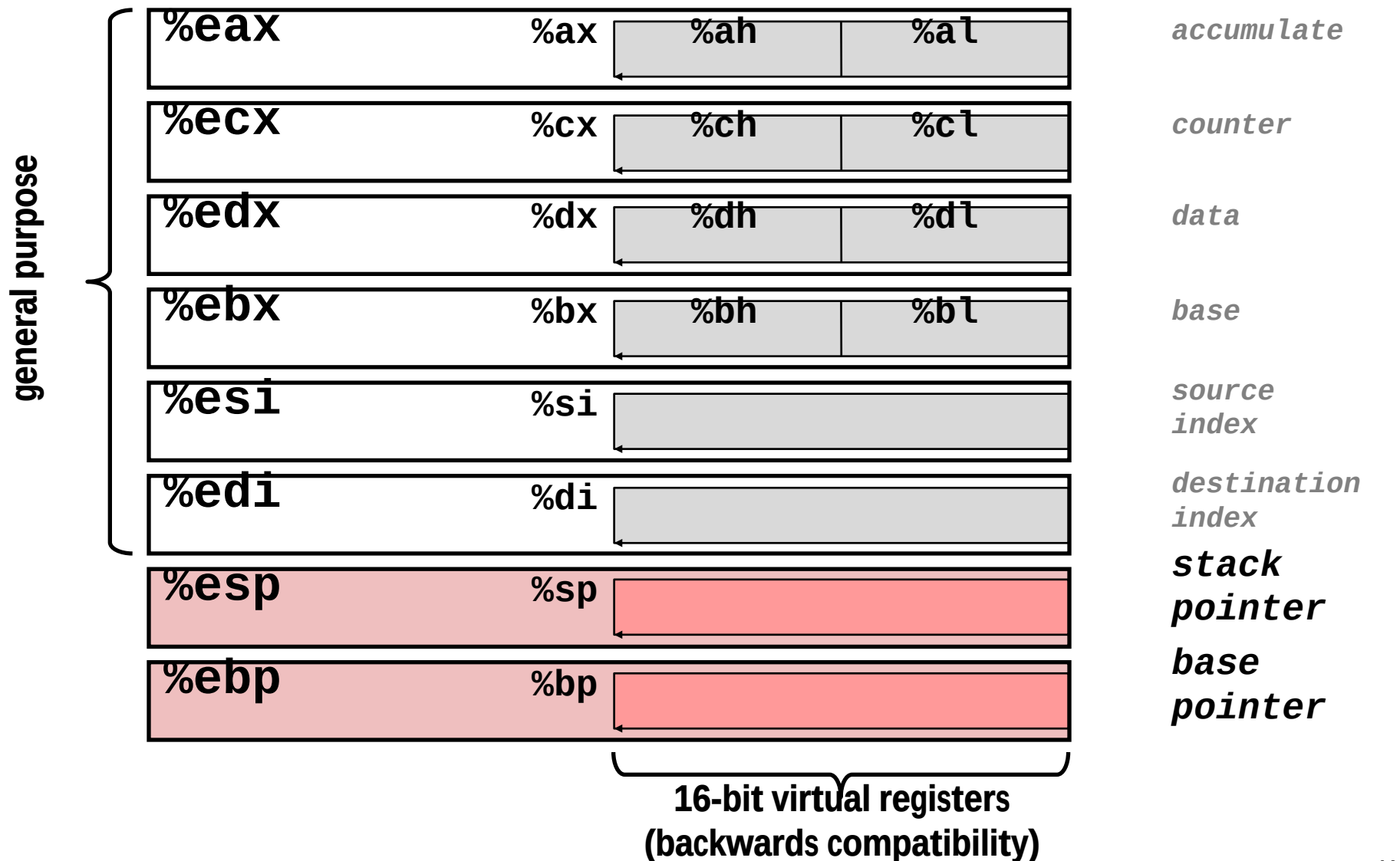
x86-64 Integer Registers

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- Can reference low-order 4 bytes (also low-order 1 & 2 bytes)
- Not part of memory (or cache)

Some History: IA32 Registers



Assembly Characteristics: Operations

- **Transfer data between memory and register**
 - Load data from memory into register
 - Store register data into memory

- **Perform arithmetic function on register or memory data**

- **Transfer control**
 - Unconditional jumps to/from procedures
 - Conditional branches
 - Indirect branches

IA-32的寻址方式

- 寻址方式
 - 根据指令给定信息得到操作数或操作数地址
- 操作数所在的位置
 - 指令中：立即寻址
 - 寄存器中：寄存器寻址
 - 存储单元中（属于存储器操作数，按字节编址）：其他寻址方式
- 存储器操作数的寻址方式与微处理器的工作模式有关
 - 两种工作模式：实地址模式和保护模式
- 实地址模式（基本用不到）
 - 为与8086/8088兼容而设，加电或复位时
 - 寻址空间为1MB，20位地址： $(CS) \ll 4 + (IP)$
- 保护模式（需要掌握）
 - 加电后进入，采用虚拟存储管理，多任务情况下隔离、保护
 - 80286以上高档微处理器最常用的工作模式
 - 寻址空间为 $2^{32}B$ ，32位线性地址分段（段基址+段内偏移量）

保护模式下的寻址方式

寻址方式	说明
立即寻址	指令直接给出操作数
寄存器寻址	指定的寄存器R的内容为操作数
位移	$LA = (SR) + A$
基址寻址	$LA = (SR) + (B)$
基址加位移	$LA = (SR) + (B) + A$
比例变址加位移	$LA = (SR) + (I) \times S + A$
基址加变址加位移	$LA = (SR) + (B) + (I) + A$
基址加比例变址加位移	$LA = (SR) + (B) + (I) \times S + A$
相对寻址	$LA = (PC) + A$

存储器操作数

跳转目标指令地址

注: LA:线性地址 (X):X的内容 SR:段寄存器 PC:程序计数器 R:寄存器
A:指令中给定地址段的位移量 B:基址寄存器 I:变址寄存器 S:比例系数

- **SR段寄存器 (间接) 确定操作数所在段的段基址**
- **有效地址**给出操作数在所在段的偏移地址

存储器操作数的寻址方式(32位)

int x;

float a[100];

short b[4][4];

char c;

long *c1;

double d[10];

a[i]的地址如何计算?

104+i×**4**

i=99时, $104 + 99 \times 4 = 500$

b[i][j]的地址如何计算?

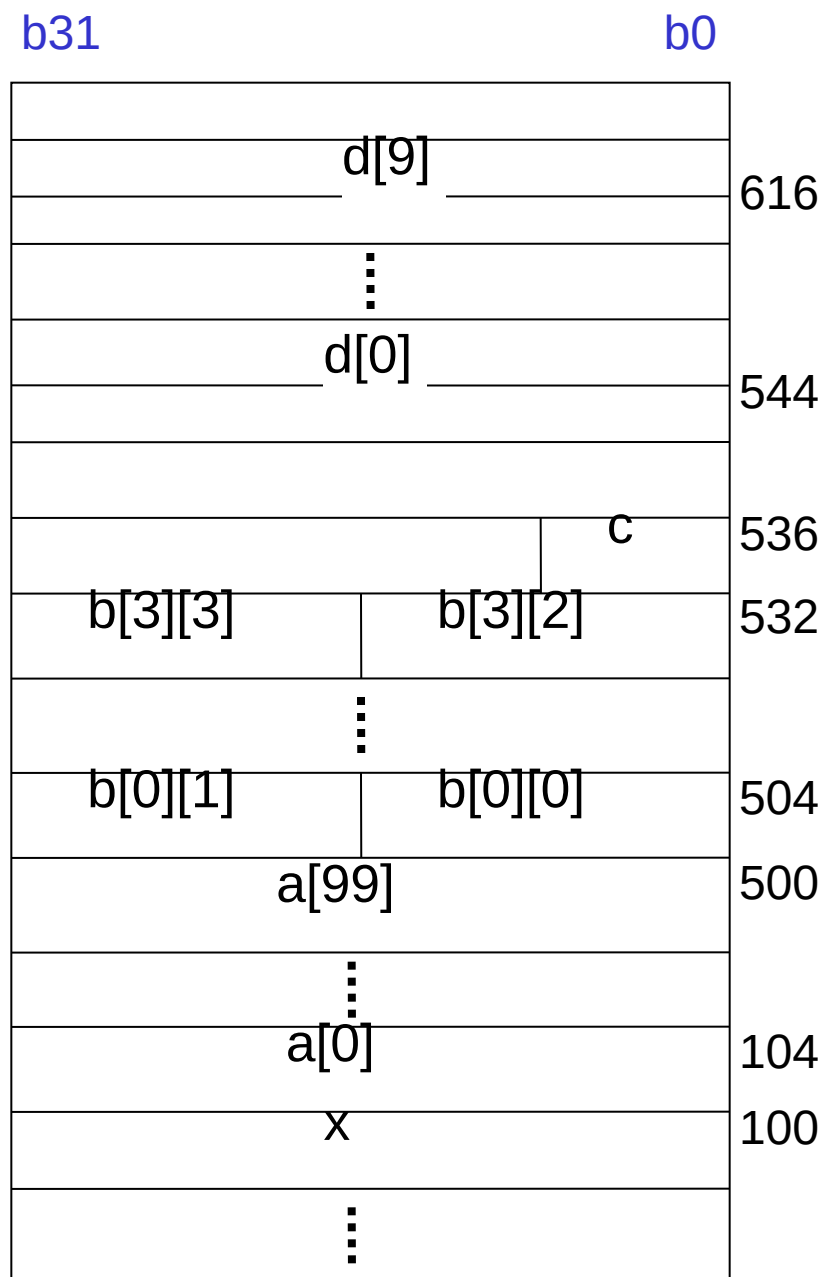
504+i×**8**+j×**2**

i=3、j=2时, $504 + 24 + 4 = 532$

d[i]的地址如何计算?

544+i×**8**

i=9时, $544 + 9 \times 8 = 616$



存储器操作数的寻址方式

```
int x;
float a[100];
short b[4][4];
char c;
long *c1;
double d[10];
```

各变量应采用什么寻址方式?

x、c: 位移 / 基址

a[i]: $104 + i \times 4$, 比例变址 + 位移

d[i]: $544 + i \times 8$, 比例变址 + 位移

b[i][j]: $504 + i \times 8 + j \times 2$,

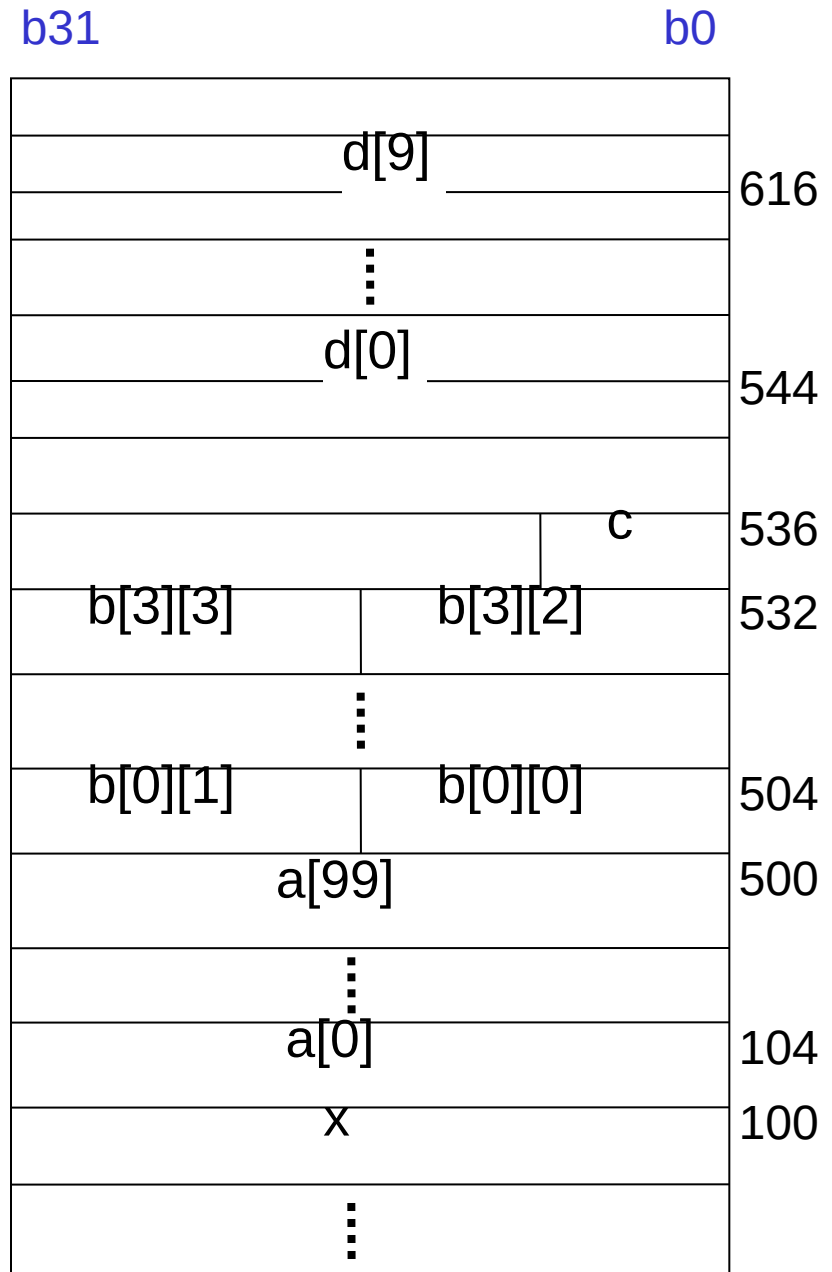
基址 + 比例变址 + 位移

将b[i][j]取到EAX中的指令可以是:

`"movw 504(%ebp,%esi,2), %eax"`

其中, $i \times 8$ 在EBP中, j在ESI中,

2为比例因子



Moving Data

■ Moving Data

`movq Source, Dest`

■ Operand Types

- **Immediate:** Constant integer data
 - Example: **\$0x400**, **\$-533**
 - Like C constant, but prefixed with ``$'`
 - Encoded with **1, 2, or 4** bytes
- **Register:** One of 16 integer registers
 - Example: `%rax`, `%r13`
 - But `%rsp` reserved for special use
 - Others have special uses for particular instructions
- **Memory:** 8 consecutive bytes of memory at address given by register
 - Simplest example: `(%rax)`
 - Various other “addressing modes”

ATT格式

C, C++ 16 制
制数必 以 0x

`%rax`

`%rcx`

`%rdx`

`%rbx`

`%rsi`

`%rdi`

`%rsp`

`%rbp`

`%rn (n=8~15)`

Warning: Intel docs use
`mov Dest, Source`

movq Operand Combinations

	Source	Dest	Src, Dest	C Analog
movq	Imm	Reg	movq \$0x4, %rax	temp = 0x4;
		Mem	movq \$-147, (%rax)	*p = -147;
	Reg	Reg	movq %rax, %rdx	temp2 = temp1;
		Mem	movq %rax, (%rdx)	*p = temp;
	Mem	Reg	movq (%rax), %rdx	temp = *p;

Cannot do memory-memory transfer with a single instruction

Simple Memory Addressing Modes

■ Normal (R) Mem[Reg[R]]

- Register R specifies memory address
- Aha! Pointer dereferencing in C

```
movq (%rcx), %rax
```

■ Displacement D(R) Mem[Reg[R]+D]

- Register R specifies start of memory region
- Constant displacement D specifies offset

```
movq 8(%rbp), %rdx
```

Example of Simple Addressing Modes

```
void  
whatAmI(<type> a, <type> b)  
{  
    ????  
}
```

%rdi

%rsi

```
whatAmI:  
    movq    (%rdi), %rax  
    movq    (%rsi), %rdx  
    movq    %rdx, (%rdi)  
    movq    %rax, (%rsi)  
    ret
```

Example of Simple Addressing Modes

```
void swap  
    (long *xp, long *yp)  
{  
    long t0 = *xp;  
    long t1 = *yp;  
    *xp = t1;  
    *yp = t0;  
}
```

```
swap:  
    movq    (%rdi), %rax  
    movq    (%rsi), %rdx  
    movq    %rdx, (%rdi)  
    movq    %rax, (%rsi)  
    ret
```

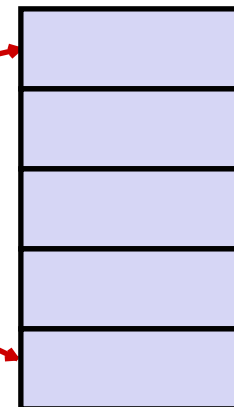

Understanding Swap()

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

Registers

%rdi	
%rsi	
%rax	
%rdx	

Memory



Register Value

%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Understanding Swap()

Registers

%rdi	0x120
%rsi	0x100
%rax	
%rdx	

Memory

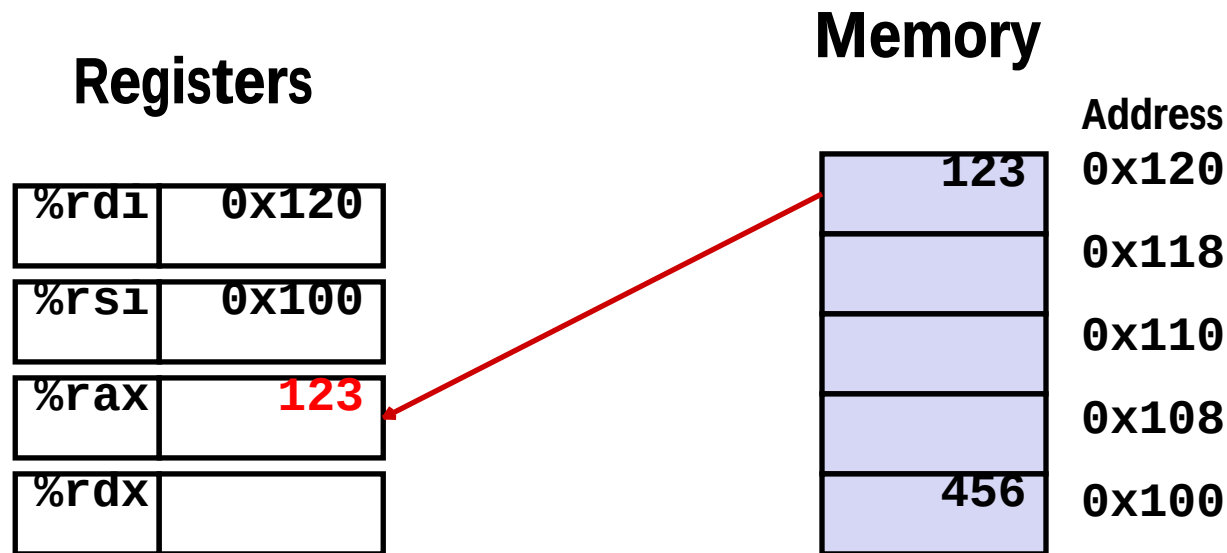
Address
123 0x120
0x118
0x110
0x108
456 0x100

swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Understanding Swap()



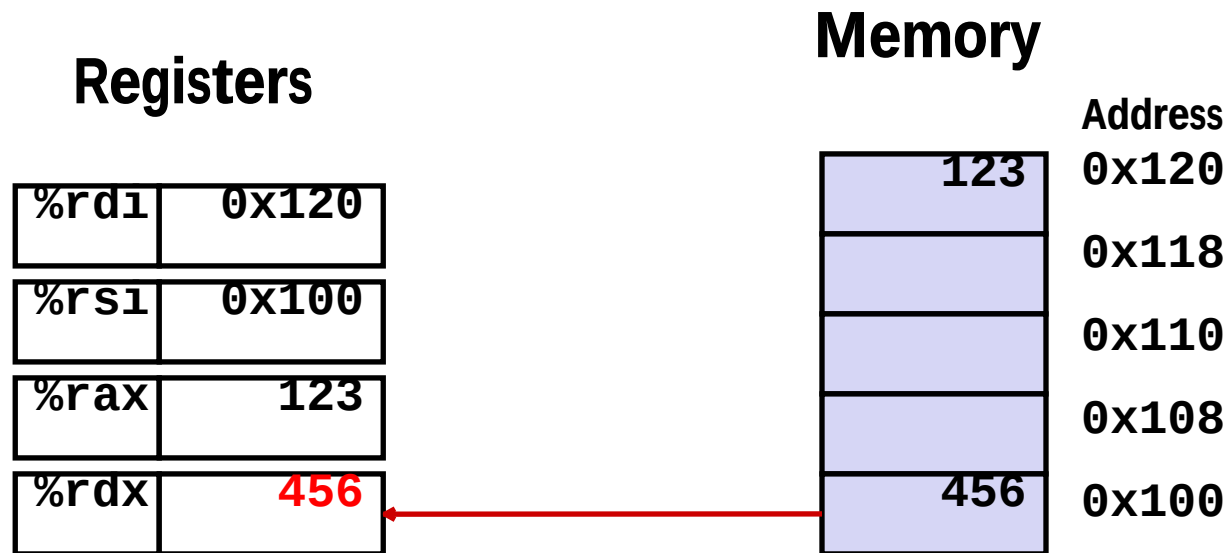
swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Understanding Swap()



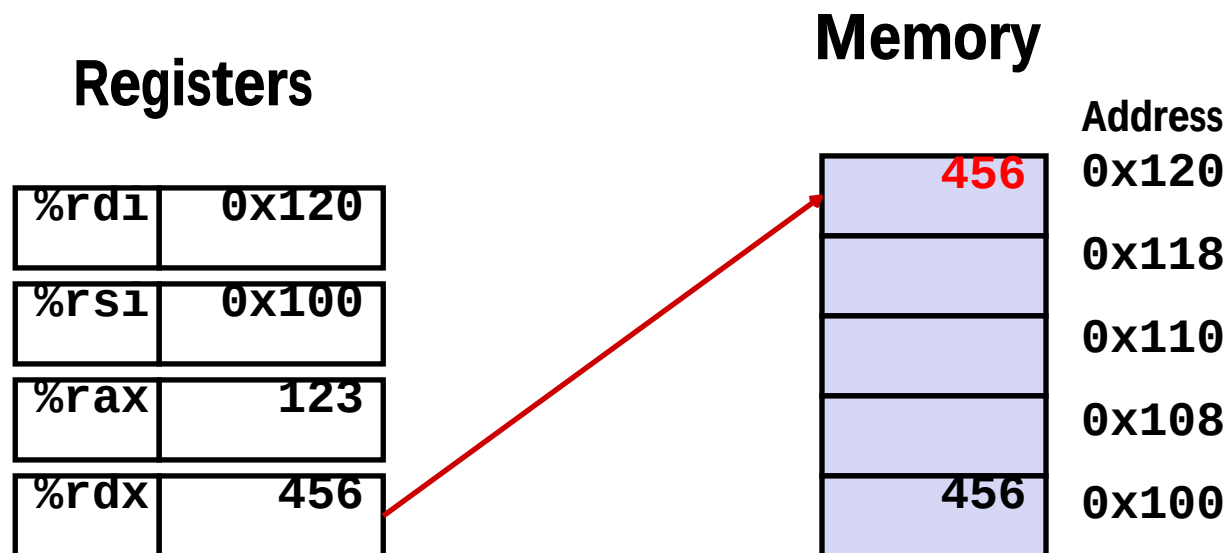
swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Understanding Swap()



swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Understanding Swap()

Registers

%rdi	0x120
%rsi	0x100
%rax	123
%rdx	456

Memory

Address	
0x120	456
0x118	
0x110	
0x108	
0x100	123



swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Simple Memory Addressing Modes

■ Normal (R) Mem[Reg[R]]

- Register R specifies memory address
- Aha! Pointer dereferencing in C

```
movq (%rcx), %rax
```

■ Displacement D(R) Mem[Reg[R]+D]

- Register R specifies start of memory region
- Constant displacement D specifies offset

```
movq 8(%rbp), %rdx
```

Complete Memory Addressing Modes

■ Most General Form

$D(Rb, Ri, S) \quad \text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri] + D]$

- D: Constant “displacement” 1, 2, or 4 bytes
- Rb: Base register: Any of 16 integer registers
- Ri: Index register: Any, **except for %rsp**
- S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

■ Special Cases

$(Rb, Ri) \quad \text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri]]$

$D(Rb, Ri) \quad \text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri] + D]$

$(Rb, Ri, S) \quad \text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri]]$

Address Computation Examples

%rdx	0xf000
%rcx	0x0100

D(Rb,Ri,S)

Mem[Reg[Rb]+S*Reg[Ri]+ D]

- D: Constant “displacement” 1, 2, or 4 bytes
- Rb: Base register: Any of 16 integer registers
- Ri: Index register: Any, except for %rsp
- S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

Expression	Address Computation	Address
0x8(%rdx)	$0xf000 + 0x8$	0xf008
(%rdx,%rcx)	$0xf000 + 0x100$	0xf100
(%rdx,%rcx,4)		
0x80(,%rdx,2)	$0 + 0xf000 + 0x80$	0xf080

Today: Machine Programming I: Basics

- History of Intel processors and architectures
- Assembly Basics: Registers, operands, move
- **Arithmetic & logical operations**
- C, assembly, machine code

IA-32常用指令类型

(1) 传送指令

— 通用数据传送指令

MOV: 一般传送, 包括movb、movw和movl等

MOVS: 符号扩展传送, 如movsbw、movswl等

MOVZ: 零扩展传送, 如movzwl、movzbl等

XCHG: 数据交换

PUSH/POP: 入栈/出栈, 如pushl, pushw, popl, popw等

— 地址传送指令

LEA: 加载有效地址, 如leal (%edx,%eax),

“%eax”的功能为 $R[edx] \leftarrow R[edx] + R[eax]$, 执行前, 若 $R[edx]=i$, $R[eax]=j$, 则指令执行后, $R[edx]=i+j$

— 输入输出指令

IN和OUT: I/O端口与寄存器之间的交换

— 标志传送指令

PUSHF、POPF: 将EFLAG压栈, 或将栈顶内容送EFLAG

IA-32常用指令类型

(2) 定点算术运算指令

- 加 / 减运算 (影响标志、不区分无/带符号)

ADD: 加, 包括addb、addw、addl等

SUB: 减, 包括subb、subw、subl等

- 增1 / 减1运算 (影响除CF以外的标志、不区分无/带符号)

INC: 加, 包括incb、incw、incl等

DEC: 减, 包括decb、decw、decl等

- 取负运算 (影响标志、若对0取负, 则结果为0/CF=0, 否则CF=1)

NEG: 取负, 包括negb、negw、negl等

- 比较运算 (做减法得到标志、不区分无/带符号)

CMP: 比较, 包括cmpb、cmpw、cmpl等

- 乘 / 除运算 (不影响标志、区分无/带符号)

MUL / IMUL: 无符号乘 / 带符号乘

DIV / IDIV: 带无符号除 / 带符号除

IA-32常用指令类型

(3) 按位运算指令

- 逻辑运算（仅NOT不影响标志，其他指令OF=CF=0，而ZF和SF根据结果设置：若全0，则ZF=1；若最高位为1，则SF=1）

NOT: 非，包括 notb、notw、notl等

AND: 与，包括 andb、andw、andl等

OR: 或，包括 orb、orw、orl等

XOR: 异或，包括 xorb、xorw、xorl等

TEST: 做“与”操作测试，仅影响标志

- 移位运算（左/右移时，最高/最低位送CF）

SHL/SHR: 逻辑左/右移，包括 shlb、shrw、shrl等

SAL/SAR: 算术左/右移，左移判溢出，右移高位补符

ROL/ROR: 循环左/右移，包括 rolb、rorw、roll等

RCL/RCR: 带循环左/右移，将CF作为操作数一部分循环移位

以上内容不要死记硬背，遇到具体指令时能查阅到并理解即可。

IA-32常用指令类型

(4) 控制转移指令

指令执行可**按顺序** 或 **跳转到转移目标指令处**执行

– 无条件转移指令

JMP DST: 无条件转移到目标指令DST处执行

– 条件转移

Jcc

DST: cc为条件码，根据标志（条件码）判断是否满足条件，若满足，则转移到目标指令DST处执行，否则按顺序执行

– 条件设置

SETcc DST: 将条件码cc保存到DST（通常是一个8位寄存器）

– 调用和返回指令（用于过程调用）

CALL DST: **返回地址RA**入栈，转DST处执行

RET: 从栈中取出返回地址RA，转到RA处执行

以上内容不要死记硬背，遇到具体指令时能查阅到并理解即可。

Address Computation Instruction

■ **leaq Src, Dst**

- Src is address mode expression
- Set Dst to address denoted by expression

■ **Uses**

- Computing addresses without a memory reference
 - E.g., translation of `p = &x[i];`
- Computing arithmetic expressions of the form $x + k*y$
 - $k = 1, 2, 4, \text{ or } 8$

■ **Example**

```
long m12(long x)
{
    return x*12;
}
```

```
leaq ($0x1000), %rax    # %rax = 1000H
leaq (%rbx), %rax       # %rax = %rbx
```

Converted to ASM by compiler:

```
leaq (%rdi,%rdi,2), %rax # t = x+2*x
salq $2, %rax           # return t<<2
```

Some Arithmetic Operations

■ Two Operand Instructions:

Format Computation

`addq Src, Dest` $\text{Dest} = \text{Dest} + \text{Src}$

`subq Src, Dest` $\text{Dest} = \text{Dest} - \text{Src}$

`imulq            Src, Dest` $\text{Dest} = \text{Dest} * \text{Src}$

`salq Src, Dest` $\text{Dest} = \text{Dest} \ll \text{Src}$

Also called `shlq`

`sarq Src, Dest` $\text{Dest} = \text{Dest} \gg \text{Src}$

Arithmetic

`shrq Src, Dest` $\text{Dest} = \text{Dest} \gg \text{Src}$

Logical

`xorq Src, Dest` $\text{Dest} = \text{Dest} \wedge \text{Src}$

`andq Src, Dest` $\text{Dest} = \text{Dest} \& \text{Src}$

`orq   Src, Dest` $\text{Dest} = \text{Dest} | \text{Src}$

- **Watch out for argument order! *Src, Dest***
(Warning: Intel docs use “op *Dest, Src*”)

- **No distinction between signed and unsigned int (why?)**

Some Arithmetic Operations

■ One Operand Instructions

`incqDest` $\text{Dest} = \text{Dest} + 1$

`decqDest` $\text{Dest} = \text{Dest} - 1$

`negqDest` $\text{Dest} = -\text{Dest}$

`notqDest` $\text{Dest} = \sim\text{Dest}$

■ See book for more instructions

Arithmetic Expression Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

```
arith:
    leaq    (%rdi,%rsi), %rax
    addq    %rdx, %rax
    leaq    (%rsi,%rsi,2), %rdx
    salq    $4, %rdx
    leaq    4(%rdi,%rdx), %rcx
    imulq    %rcx, %rax
    ret
```



Interesting Instructions

- **leaq**: address computation
- **salq**: shift
- **imulq**: multiplication
 - But, only used once

Understanding Arithmetic Expression

Example

```
long arith
(long x, long y, long z)
{
    long t1 = x+y;
    long t2 = z+t1;
    long t3 = x+4;
    long t4 = y * 48;
    long t5 = t3 + t4;
    long rval = t2 * t5;
    return rval;
}
```

```
arith:
    leaq    (%rdi,%rsi), %rax    # t1
    addq    %rdx, %rax          # t2
    leaq    (%rsi,%rsi,2), %rdx
    salq    $4, %rdx            # t4
    leaq    4(%rdi,%rdx), %rcx   # t5
    imulq   %rcx, %rax          # rval
    ret
```

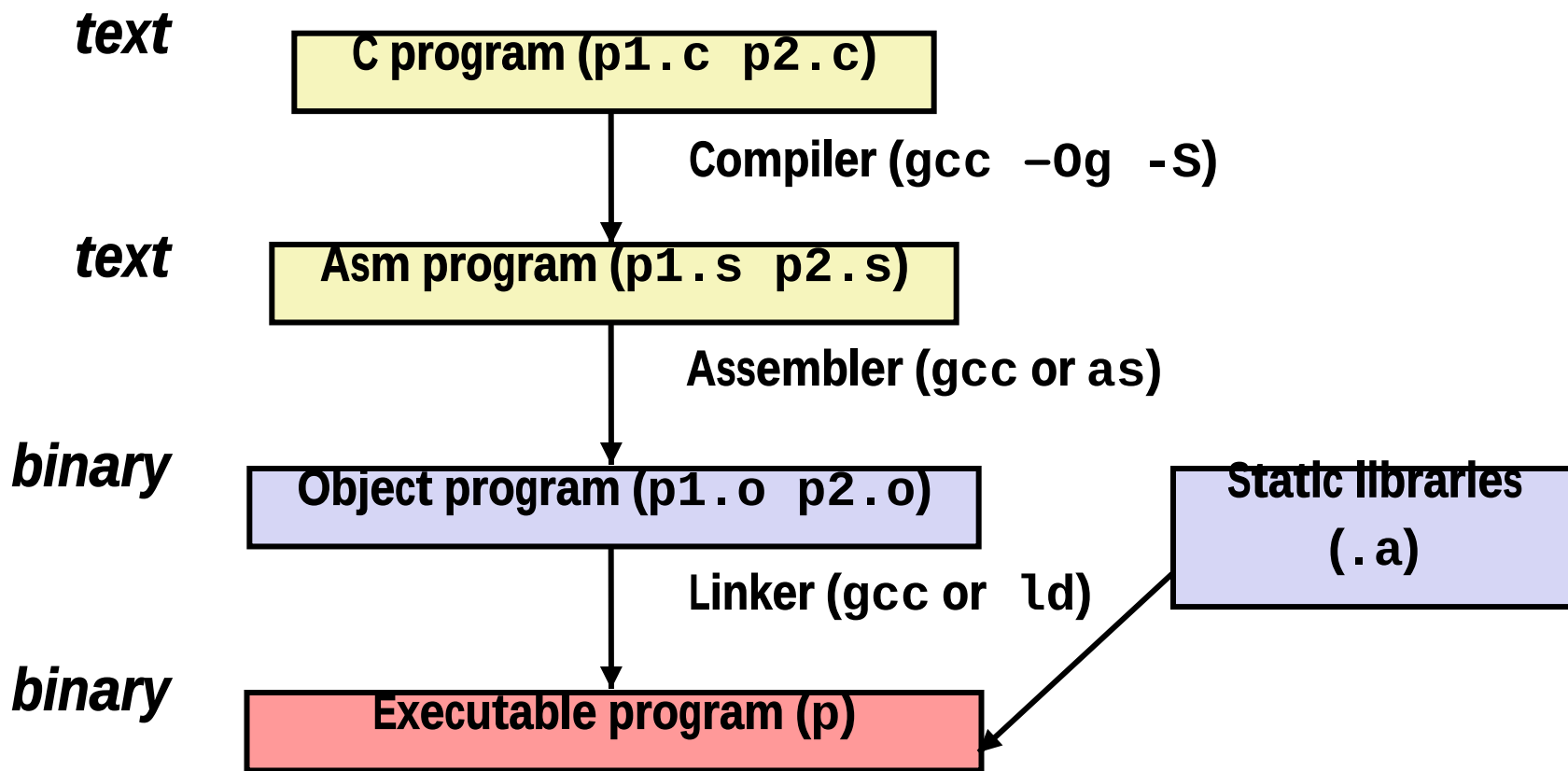
Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z, t4
%rax	t1, t2, rval
%rcx	t5

Today: Machine Programming I: Basics

- History of Intel processors and architectures
- Assembly Basics: Registers, operands, move
- Arithmetic & logical operations
- **C, assembly, machine code**

Turning C into Object Code

- Code in files **p1.c** **p2.c**
- Compile with command: **gcc -Og p1.c p2.c -o p**
 - Use basic optimizations (**-Og**) [New to recent versions of GCC]
 - Put resulting binary in file **p**



Compiling Into Assembly

C Code (sum.c)

```
long plus(long x, long y);  
  
void sumstore(long x, long y,  
              long *dest)  
{  
    long t = plus(x, y);  
    *dest = t;  
}
```

Generated x86-64 Assembly

```
sumstore:  
    pushq    %rbx  
    movq     %rdx, %rbx  
    call     plus  
    movq     %rax, (%rbx)  
    popq     %rbx  
    ret
```

Obtain (on shark machine) with command

```
gcc -Og -S sum.c
```

Produces file `sum.s`

Warning: Will get very different results on non-Shark machines (Andrew Linux, Mac OS-X, ...) due to different versions of gcc and different compiler settings.

What it really looks like

```
.globl sumstore
.type sumstore, @function
sumstore:
.LFB35:
.cfi_startproc
pushq %rbx
.cfi_def_cfa_offset 16
.cfi_offset 3, -16
movq %rdx, %rbx
call plus
movq %rax, (%rbx)
popq %rbx
.cfi_def_cfa_offset 8
ret
.cfi_endproc
.LFE35:
.size sumstore, .-sumstore
```

What it really looks like

```

        .globl  sumstore
        .type   sumstore, @function
sumstore:
.LFB35:
        .cfi_startproc
        pushq   %rbx
        .cfi_def_cfa_offset 16
        .cfi_offset 3, -16
        movq    %rdx, %rbx
        call    plus
        movq    %rax, (%rbx)
        popq    %rbx
        .cfi_def_cfa_offset 8
        ret
        .cfi_endproc
.LFE35:
        .size   sumstore, .-sumstore

```

Things that look weird
and are preceded by a '.'
are generally directives.

```

sumstore:
    pushq   %rbx
    movq    %rdx, %rbx
    call    plus
    movq    %rax, (%rbx)
    popq    %rbx
    ret

```


GCC使用举例

```

1 // test.c
2 #include <stdio.h>
3 int add(int i, int j)
4 {
5     int x = i + j;
6     return x;
7 }

```

test1.c和test2.c, 最终生成可执行文件为test
c test2.c -o test

级优化, -O2为二级优化, 选项-o指出输出文件名
'objdump -d test.o" 反汇编为汇编语言程序

gcc -E test.c -o test.i
gcc -S test.i -o test.s

gcc -S test.c -o test.s

test.s

```

add:
pushl %ebp
movl %esp, %ebp
subl $16, %esp
movl 12(%ebp), %eax
movl 8(%ebp), %edx
leal (%edx, %eax), %eax
movl %eax, -4(%ebp)
movl -4(%ebp), %eax
leave
ret

```

00000000 <add>:

0:	55	push %ebp
1:	89 e5	mov %esp, %ebp
3:	83 ec 10	sub \$0x10, %esp
6:	8b 45 0c	mov 0xc(%ebp), %eax
9:	8b 55 08	mov 0x8(%ebp), %edx
c:	8d 04 02	leal (%edx,%eax,1), %eax
f:	89 45 fc	mov %eax, -0x4(%ebp)
12:	8b 45 fc	mov -0x4(%ebp), %eax
15:	c9	leave
16:	c3	ret

位移量

机器指令

汇编指令

编译得到的与反汇编得到的汇编指令形式稍有差异

两种目标文件

```

1 // test.c
2 #include <stdio.h>
3 int add(int i, int j)
4 {
5     int x = i + j;
6     return x;
7 }

```

test.o: 可重定位目标文件

test: 可执行目标文件

“objdump -d test.o” 结果

00000000 <add>:

0:	55	push	%ebp
1:	89 e5	mov	%esp, %ebp
3:	83 ec 10	sub	\$0x10, %esp
6:	8b 45 0c	mov	0xc(%ebp), %eax
9:	8b 55 08	mov	0x8(%ebp), %edx
c:	8d 04 02	lea	(%edx,%eax,1), %eax
f:	89 45 fc	mov	%eax, -0x4(%ebp)
12:	8b 45 fc	mov	-0x4(%ebp), %eax
15:	c9	leave	
16:	c3	ret	

“objdump -d test” 结果

080483d4 <add>:

80483d4:	55	push ...
80483d5:	89 e5	...
80483d7:	83 ec 10	...
80483da:	8b 45 0c	...
80483dd:	8b 55 08	...
80483e0:	8d 04 02	...
80483e3:	89 45 fc	...
80483e6:	8b 45 fc	...
80483e9:	c9	...
80483ea:	c3	ret

test.o中的代码从地址0开始， test中的代码从80483d4开始!

Object Code

Code for sumstore

0x0400595:

0x53

0x48

0x89

0xd3

0xe8

0xf2

0xff

0xff

0xff

0x48

0x89

0x03

0x5b

0xc3

- Total of 14 bytes

- Each instruction 1, 3, or 5 bytes

- Starts at address 0x0400595

■ Assembler

- Translates .s into .o
- Binary encoding of each instruction
- Nearly-complete image of executable code
- Missing linkages between code in different files

■ Linker

- Resolves references between files
- Combines with static run-time libraries
 - E.g., code for **malloc**, **printf**
- Some libraries are *dynamically linked*
 - Linking occurs when program begins execution

Machine Instruction Example

```
*dest = t;
```

```
movq %rax, (%rbx)
```

```
0x40059e: 48 89 03
```

■ C Code

- Store value **t** where designated by **dest**

■ Assembly

- Move 8-byte value to memory
 - Quad words in x86-64 parlance
- Operands:

t: Register **%rax**

dest: Register **%rbx**

***dest:** Memory **M[%rbx]**

■ Object Code

- 3-byte instruction
- Stored at address **0x40059e**

Disassembling Object Code

Disassembled

```

0000000000400595 <sumstore>:
 400595: 53                push    %rbx
 400596: 48 89 d3          mov     %rdx,%rbx
 400599: e8 f2 ff ff ff    callq   400590 <plus>
 40059e: 48 89 03          mov     %rax, (%rbx)
 4005a1: 5b                pop     %rbx
 4005a2: c3                retq

```

■ Disassembler

objdump -d sum

- Useful tool for examining object code
- Analyzes bit pattern of series of instructions
- Produces approximate rendition of assembly code
- Can be run on either a .out (complete executable) or .o file

Alternate Disassembly

Disassembled

Dump of assembler code for function `sumstore`:

```
0x0000000000400595 <+0>: push    %rbx
0x0000000000400596 <+1>: mov     %rdx,%rbx
0x0000000000400599 <+4>: callq  0x400590 <plus>
0x000000000040059e <+9>: mov     %rax, (%rbx)
0x00000000004005a1 <+12>: pop     %rbx
0x00000000004005a2 <+13>: retq
```

■ Within gdb Debugger

- Disassemble procedure

`gdb sum`

`disassemble sumstore`

Alternate Disassembly

Object Code

0x0400595:

0x53

0x48

0x89

0xd3

0xe8

0xf2

0xff

0xff

0xff

0x48

0x89

0x03

0x5b

0xc3

Disassembled

Dump of assembler code for function sumstore:

0x0000000000400595 <+0>: push %rbx

0x0000000000400596 <+1>: mov %rdx,%rbx

0x0000000000400599 <+4>: callq 0x400590 <plus>

0x000000000040059e <+9>: mov %rax, (%rbx)

0x00000000004005a1 <+12>: pop %rbx

0x00000000004005a2 <+13>: retq

■ Within gdb Debugger

- Disassemble procedure

gdb sum

disassemble sumstore

- Examine the 14 bytes starting at sumstore

x/14xb sumstore

What Can be Disassembled?

```
% objdump -d WINWORD.EXE
```

```
WINWORD.EXE: file format pei-i386
```

```
No symbols in "WINWORD.EXE".
```

```
Disassembly of section .text:
```

```
30001000 <.text>:
```

```
30001000: 55          push    %ebp
```

```
30001001:
```

```
30001003:
```

```
30001005:
```

```
3000100a:
```

**Reverse engineering forbidden by
Microsoft End User License Agreement**

- Anything that can be interpreted as executable code
- Disassembler examines bytes and reconstructs assembly source

Machine Programming I: Summary

■ History of Intel processors and architectures

- Evolutionary design leads to many quirks and artifacts

■ C, assembly, machine code

- New forms of visible state: program counter, registers, ...
- Compiler must transform statements, expressions, procedures into low-level instruction sequences

■ Assembly Basics: Registers, operands, move

- The x86-64 move instructions cover wide range of data movement forms

■ Arithmetic

- C compiler will figure out different instruction combinations to carry out computation